

0. Lambda - Ek anonymous function hai jo bina naam ke, bina return type likhe short syntax mein likha ja sakta hai, mainly functional interfaces (single abstract method wale interfaces) ko implement karne ke liye use hota hai. Syntax: (parameters) -> expression. Example: (a, b) -> a + b. Ye boilerplate code (anonymous class likhna) kaafi kam kar deta hai.

2. Parallel Stream - Normal stream ki tarah hi hai, bas multiple threads (CPU cores) use karke data process karta hai, jisse large datasets pe processing fast ho jati hai. stream() ki jagah parallelStream() likhna hota hai.

1. Streams - Can only be used once, After use they are gone. Ek pipeline hai jo collections (List, Set, etc.) pe data process karne ke liye use hota hai, functional style mein — declarative way mein "kya karna hai" likhte ho, "kaise karna hai" JVM khud handle karta hai. Stream khud data store nahi karta, source (jaise List) se data leke transformations apply karta hai aur result deta hai. Ek baar consume hone ke baad stream reuse nahi ho sakta.



Spring Boot Web and MVC

Chhote datasets ke liye avoid karo kyunki thread management ka overhead fayde se zyada ho sakta hai.

Homework

3. default method Interface ke andar method body ke saath likha gaya method, jisse implementing classes ko wo method mandatory implement nahi karna padta (optional ho jata hai). Ye backward compatibility maintain karne ke liye introduce hua tha — purani interfaces mein naye methods add karne ke liye bina existing implementations ko break kiye. Keyword: default. Example: ConstraintValidator ka initialize() method default hai.

4. stream(), foreach(), sorted(), map() - to convert into other type, filter, collect - Ye sab Stream ke common operations hain. stream() collection ko stream mein convert karta hai; forEach() har element pe action perform karta hai (terminal operation); sorted() elements ko order mein arrange karta hai; map() har element ko transform/convert karke naye type mein badalta hai (jaise Entity → DTO); filter() condition ke basis pe elements select karta hai (jo match kare wahi rahe); collect() stream ko wapas List/Set/Map jaisi collection mein convert karta hai (terminal operation).

5. Chaining - Some stream method returns another stream hence other stream method can be used in chain manner for processing. Multiple methods ko ek ke baad ek . (dot) se jodna, jisse code readable aur concise ban jata hai — har method call previous result pe kaam karta hai. Ye Stream API, Builder pattern, aur Optional sab mein use hota hai. Example: list.stream().filter(x -> x > 5).map(x -> x * 2).collect(Collectors.toList()) — sab ek chain mein.

Recap

1. Spring Boot Web and MVC Architecture
2. Presentation Layer
3. Persistence Layer
4. Service Layer
5. Exception Handling in Spring Boot MVC
6. Input Validation Annotations

6. Collectors , collect() - collect() ek terminal operation hai jo stream ko final result (List, Set, Map, String, etc.) mein convert karta hai. Collectors ek utility class hai jo ready-made collection strategies provide karti hai jaise Collectors.toList(), Collectors.toSet(), Collectors.joining(), Collectors.groupingBy(). collect() ke andar Collectors ka koi method pass karke batate ho ki result kis form mein chahiye.

7. :: (Method Reference) — Lambda expression ka short-hand syntax hai jab lambda sirf ek existing method ko call kar raha ho, kuch aur extra kaam nahi kar raha. Syntax: ClassName::methodName.
Example: (x) -> x.toString() ki jagah Object::toString likh sakte ho. 4 types hote hain: static method (ClassName::staticMethod), instance method of particular object (obj::method), instance method of arbitrary object (ClassName::instanceMethod), aur constructor reference (ClassName::new).

Homework

1. Make a list of all the annotations we have seen so far in the Spring Web Framework.
2. Create the following REST endpoints for the following Entity

@ExceptionHandler — Specific exception type ke liye handler method define karta hai, jo us exception ko catch karke custom response deta hai.

@RestControllerAdvice — Global exception handling aur response customization ke liye, poore application ke controllers pe apply hota hai.

Department

- id
- title
- isActive
- createdAt

REST APIs:

GET: /departments

POST: /departments

PUT: /departments

DELETE: /departments

GET: /departments/{id}

@Data — Lombok annotation, Getter + Setter + toString() + equals() + hashCode() sab ek sath generate kar deta hai.

@Builder — Lombok annotation, Builder Design Pattern automatically generate karta hai (fluent object creation ke liye).

@JsonFormat — Field (jaise Date/LocalDateTime) ka JSON mein serialize/deserialize hone ka specific pattern/format define karta hai.

@FunctionalInterface — Interface ko mark karta hai ki usme sirf ek hi abstract method hona chahiye (lambda expressions ke liye use hota hai); compiler enforce karta hai ye rule.

Homework

3. Write Exception Handling logic for the Department Entity.

4. Add the appropriate fields in the Department and Employee Entities so that the following validations can be added:

`@Null`, `@NotNull`, `@AssertTrue`, `@AssertFalse`, `@Min`, `@Max`, `@DecimalMin`,
`@DecimalMax`, `@Negative`, `@NegativeOrZero`, `@Positive`, `@PositiveOrZero`,
`@Size`, `@Digits`, `@Past`, `@PastOrPresent`, `@Future`, `@FutureOrPresent`,
`@Pattern`, `@Email`, `@NotEmpty`, `@NotBlank`, `@Length`, `@Range`,
`@CreditCardNumber`, `@URL`

`@RequiredArgsConstructor` — Lombok annotation, sirf final ya `@NonNull` fields ka constructor generate karta hai (dependency injection ke liye best practice).

`@Valid` — Request body ya object pe validation annotations (`@NotBlank`, `@Email`, etc.) ko trigger karta hai.

interface — Java ka blueprint/contract jisme abstract methods (aur optionally default/static methods) hote hain, jinhe implementing class ko implement karna padta hai.

`@Target` — Custom annotation ke liye specify karta hai ki wo kaha (FIELD, METHOD, TYPE, etc.) apply ho sakta hai.

`@Retention` — Custom annotation ki information kab tak retain (SOURCE, CLASS, RUNTIME) rakhni hai, ye define karta hai.

`@Constraint` — Custom validation annotation ko batata hai ki uska validation logic kaunsi `ConstraintValidator` class mein hai.

Homework

5. Create custom Annotation to handle a Prime number input
6. Create custom Annotation to check if the Password String field satisfies the following criteria:
 - a. contains one uppercase letter
 - b. contains one lowercase letter
 - c. contains one special character
 - d. minimum length is 10 characters

@NoArgsConstructor — Lombok annotation, bina kisi parameter wala (empty) constructor generate karta hai.

@Service — Class ko Service layer ka Spring bean mark karta hai; business logic yahan likhi jaati hai.

@Bean — Method ke return value ko Spring container mein manually ek bean ki tarah register karta hai (Configuration class ke andar use hota hai).

@Configuration — Class ko Spring configuration class mark karta hai, jisme @Bean methods define hote hain.

@DeleteMapping — DELETE HTTP request handle karta hai; resource delete karne ke liye use hota hai.

@PatchMapping — PATCH HTTP request handle karta hai; resource ka partial (kuch fields) update karne ke liye use hota hai.

